

Bucharest University of Economic Studies

11001
10001 11100110
0010 110001 11000110
00101001 01011010 1100
010101 1100000100 100
00011111 10 1001110
00101 11010 10
10010 101
00100
01001
00110
0000110

Data Structure

Assoc. prof. Mihai DOINEA, PhD

mihai.doinea@ie.ase.ro



Assessment

- Final exam: 60%
 - Written exam: 60% (50% minimum passing grade)
- Laboratory grading: 40%
 - Ongoing evaluation: 10%
 - Practical test: 15%
 - Practical test: 15%
- Website:
 - <http://online.ase.ro>
 - <https://github.com/mihaidoinea/DS2022>



Resources

- Ion IVAN, Marius POPA, Paul POCATILU, Mihai DOINEA – *Structuri de date*, Editura ASE, Bucuresti, 2008, ISBN 978-606-505-031-0
- Ion SMEUREANU – *Programarea in limbajul C/C++*, Editura CISON, 2001
- Donald KNUTH – *The Art of Computer Programming, Fundamental Algorithms* (3rd edition, 2021), Addison Wesley Printing House, 1997, ISBN: 978-0-201-89683-1
- Michael MAIN – *Data Structures & Other Objects Using Java*, Pearson Publisher, 2012, ISBN: 9780132911504



Content – Fundamentals

1. Pointers

- a. Stack vs. Heap
- b. Parameters passing

2. Bits structures

3. Memory classes

4. Union structure



Content – Linear structures

1. Dynamic data structures – Linked and double linked lists
2. Stack and Queue
3. Hash Table – array of lists
4. Heap – complete trees, array implementation
5. Collections Maps & Sets



Content – Nonlinear structures

1. Binary Search Trees

2. Binary Balanced Trees vs. Imbalanced Trees

3. B trees

4. Graphs



Content – Applications

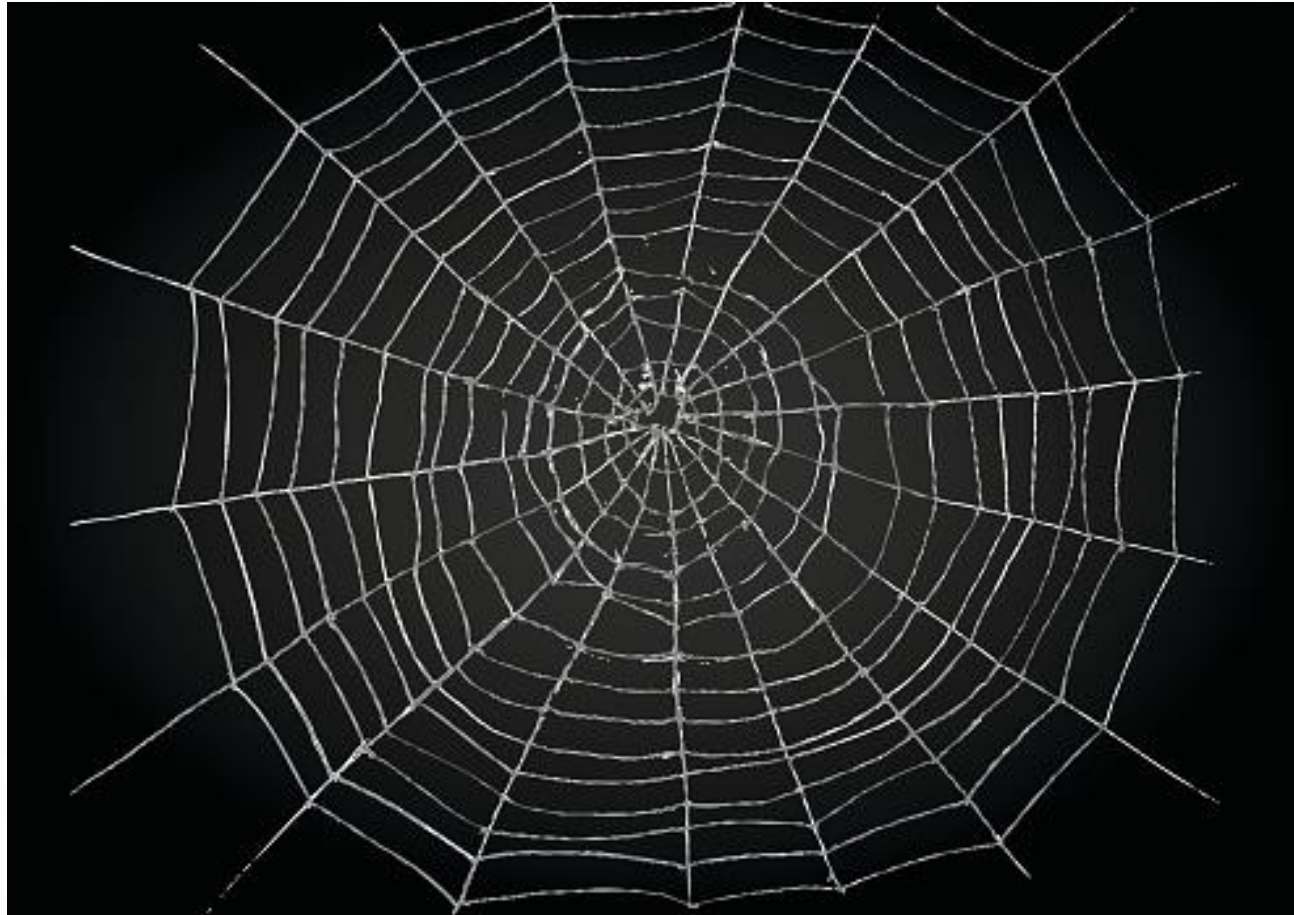
1. Compression
2. String processing
3. Sorting
4. Conversions
5. Library functions



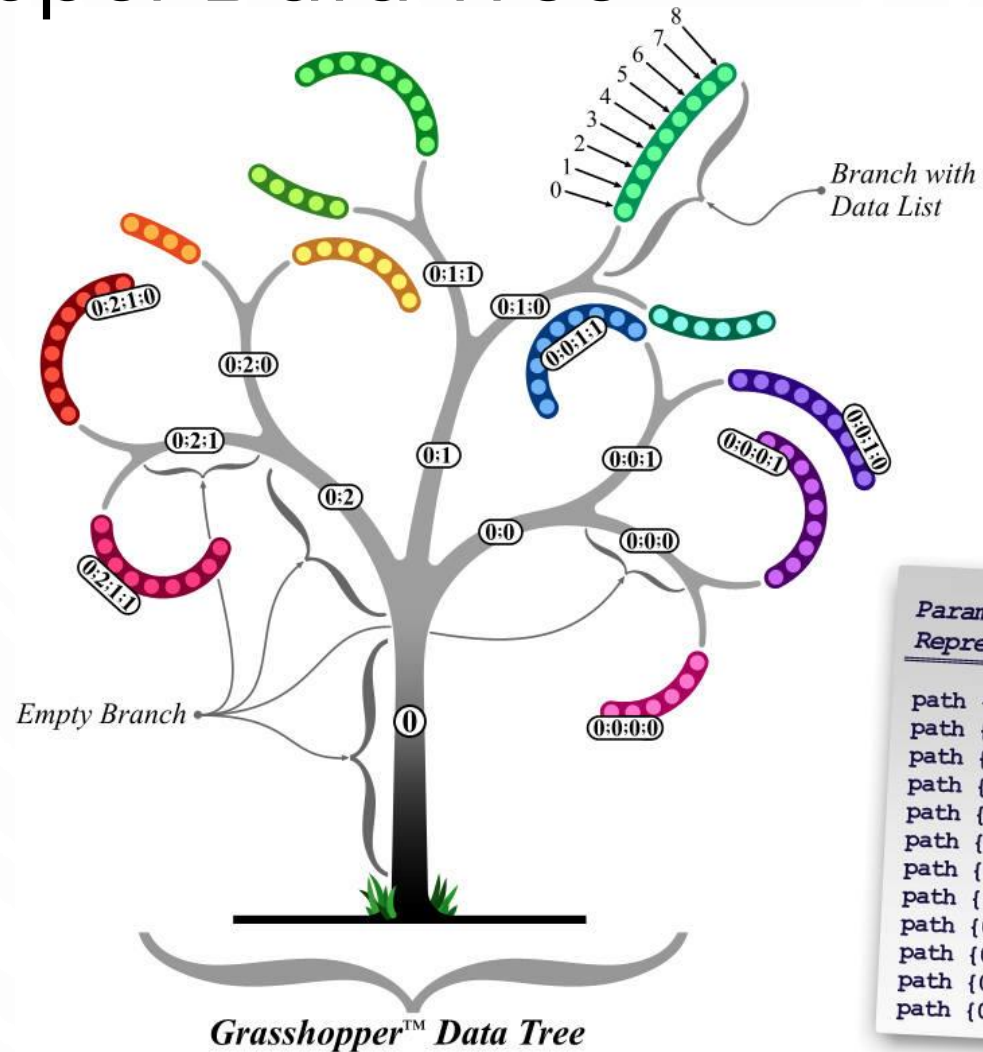
Art Structure



Spider web



Grasshopper Data Tree



Param Viewer Representation

```
path {0;0;0;0} (N = 6)
path {0;0;0;1} (N = 9)
path {0;0;1;0} (N = 9)
path {0;0;1;1} (N = 9)
path {0;1;0;0} (N = 6)
path {0;1;0;1} (N = 9)
path {0;1;1;0} (N = 9)
path {0;1;1;1} (N = 5)
path {0;2;0;0} (N = 7)
path {0;2;0;1} (N = 4)
path {0;2;1;0} (N = 9)
path {0;2;1;1} (N = 9)
```



Data Structures

- Instrument for data usage which helps to create information;
- A method of organizing and storing units of data in the external and internal memory;
- A collection of variables stored in a predefined state in memory;



Data Structures

Relationship among data elements that dictates how data is recorded, manipulated, stored and presented.



Characteristics

Dynamic vs Static

- Dynamic – grow and shrink at runtime
- Static – fixed sizes but greater performance

Memory layout

- Contiguous – compact memory layout
- Scattered – stored across multiple regions

Algorithm complexity

- Big O notation

Optimization

- Memory optimization
- Time optimization



Necessity - Algorithm efficiency



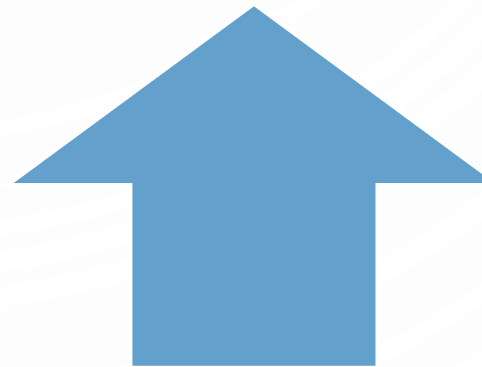
Time complexity

- Execution time (machine cycles)
- Processing depth

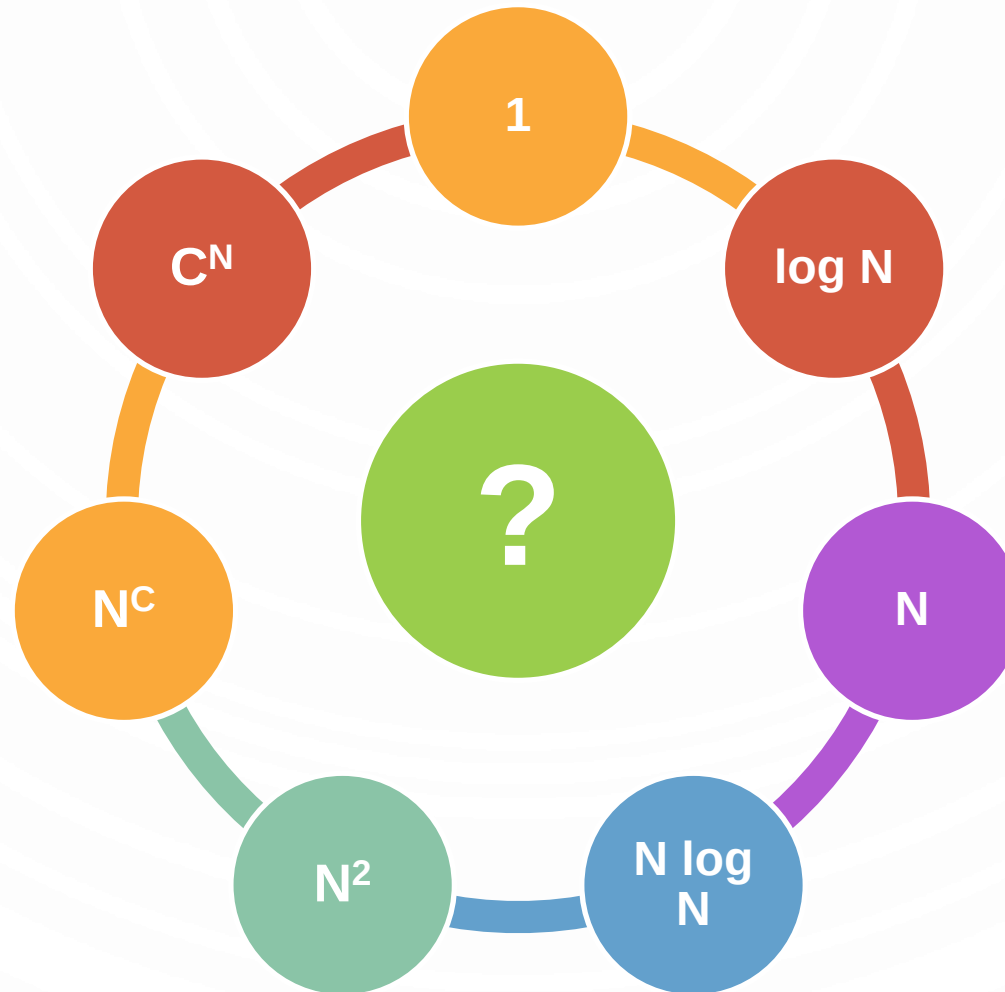


Space complexity

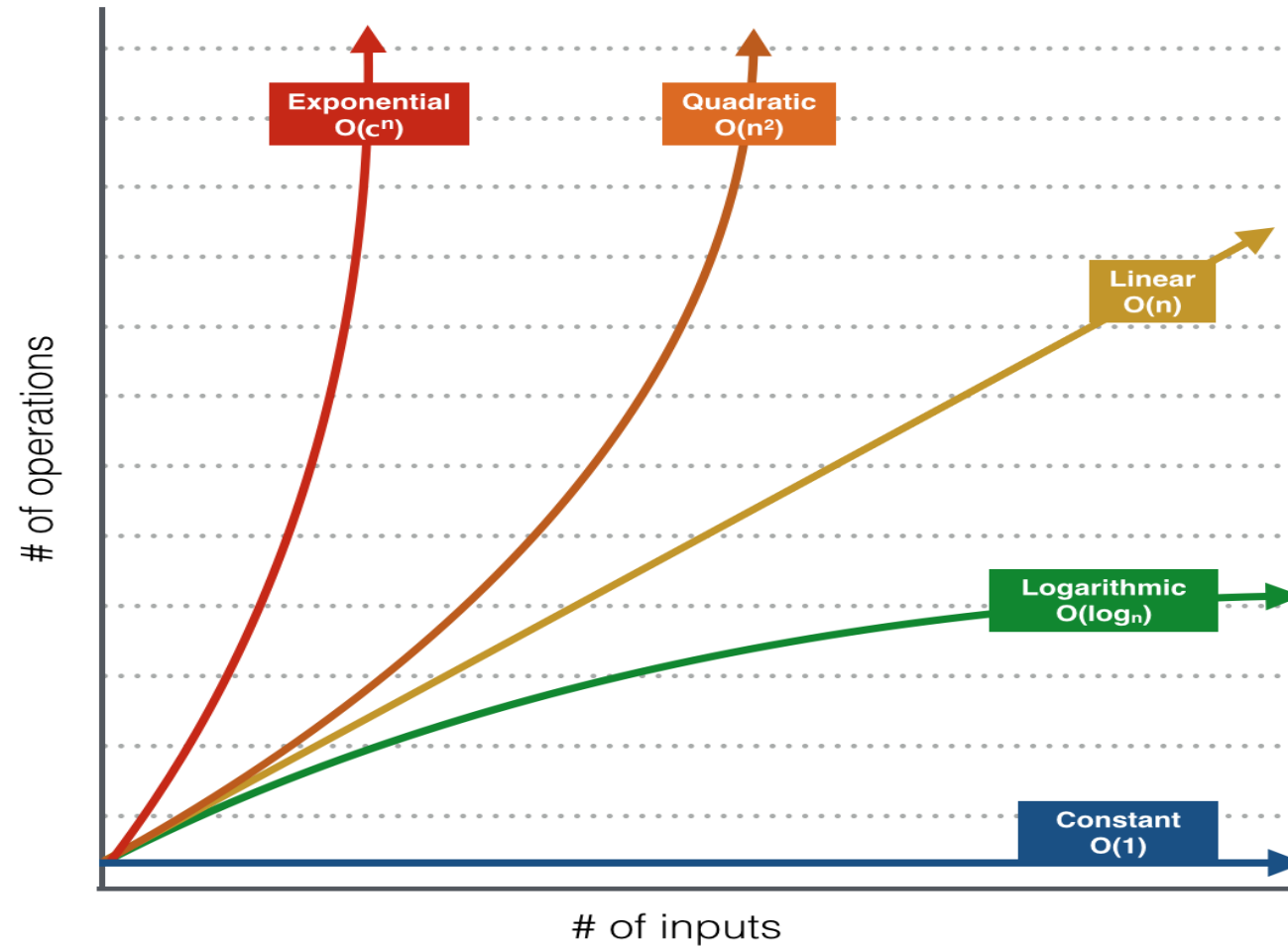
- Memory usage (bytes)
- Data input size
- Static space + Dynamic space



Big-O Analysis – Complexity classes



Big-O Analysis



Good code description

Runs faster

Requires
less
memory

Readable

Gives
correct
output

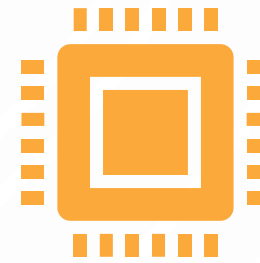


How to define good code?



Time complexity

How fast the algorithm runs giving good results;



Memory complexity

How much memory does the algorithm requires to deliver good results;



Memory – Structure



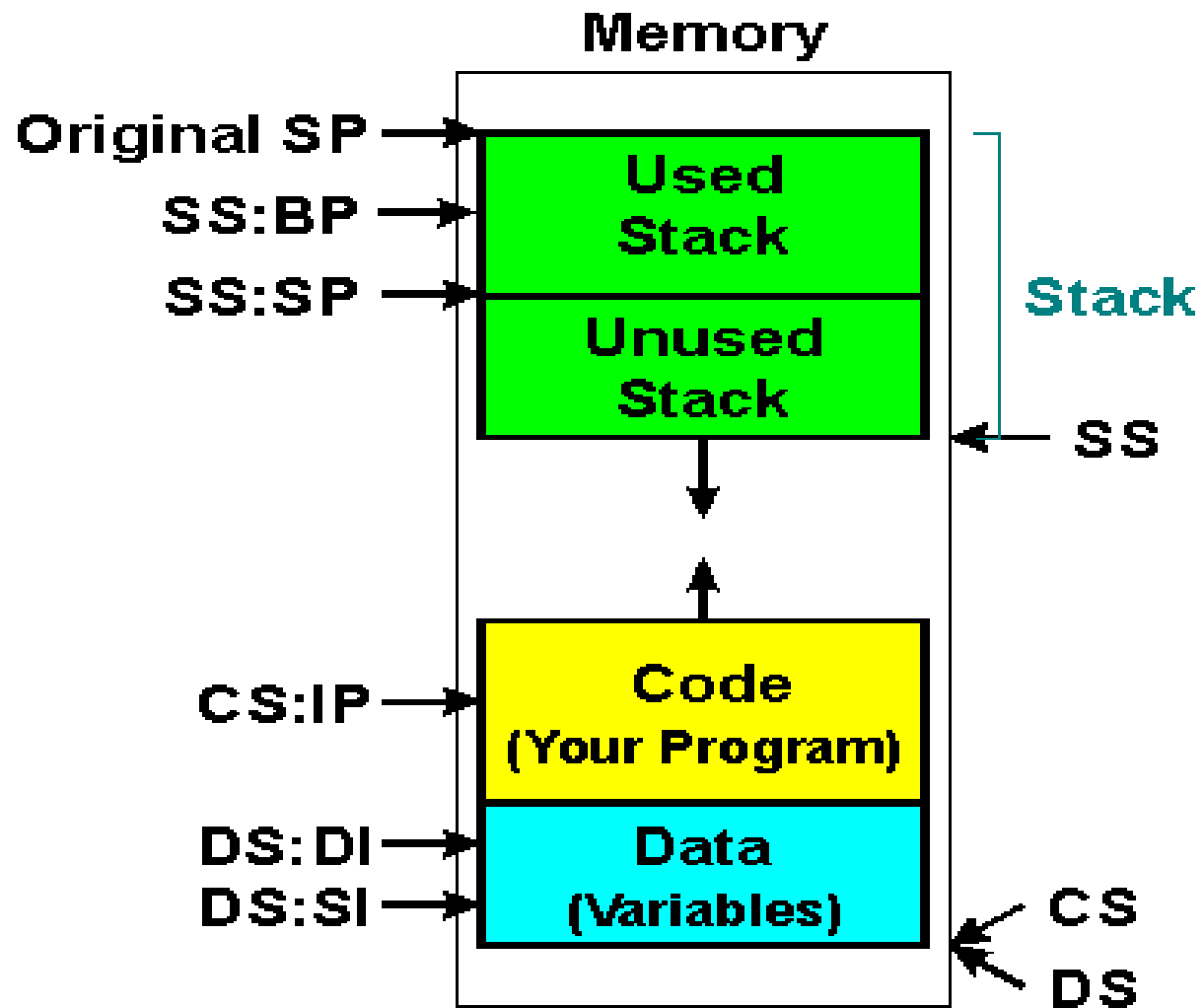
Memory – Intel 8086 Registers

Segment registers:

- Data segment: DS
- Stack segment: SS
- Code segment: CS



Stack



Function comparisons

C/C++ Source

```
#include<stdio.h>

void main()
{
    static char a = 7;
    static char b = 9;
    static int c = 0;
    c = a+b;
}
```

Assembly Language

```
.model small
.stack 16
.data

a db 7
b db 9
c dw 0

code
start:

    mov AX, @data
    mov DS, AX

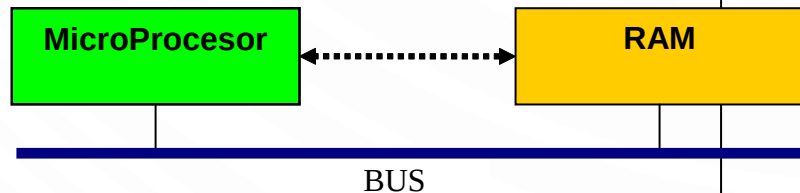
    mov AL,a
    add AL,b
    mov c,AX

    mov AX, 4C00h
    int 21h

end start
```

Machine code

```
B8 02 00 8E D8
A0 00 00 02 06
01 00 A3 02 00
B8 00 4C CD 21
00 00 00.....00 00
07 09
```



Pointers

Variable whose value is an address that points to a different memory area;

The address size is given by the processor architecture;



Pointers

Pointer variable utility

- Source code optimization / memory allocation
- Returning values as result of a function call
- Implementing arrays and arrays of characters



Pointers

Definition:

```
type *namePtr;
```

Initialization:

```
namePtr = &nameVar;  
namePtr = NULL;
```

Usage:

```
type nameVar = *namePtr;
```



Pointers – Example

```
int      *ip; // integer pointer
```

```
double  **dp; // pointer to pointer to double
```

```
float   *vfp[10]; // array of 10 pointers to float
```

```
char     *ch; // pointer to a char
```

```
int (*suma)(lista_param); //pointer to a function
```



Pointers – Values

Pointer's values

`tip *p1;`

`0xffffffffch`

`tip *p1=NULL;`

`0x00000000h`

`tip *p1=&a;`

`0x00ABF23Ch`



Pointers – Applications

Functions parameter passing

Call by
value

Call by address

Reference
(&)

Pointers
(*)

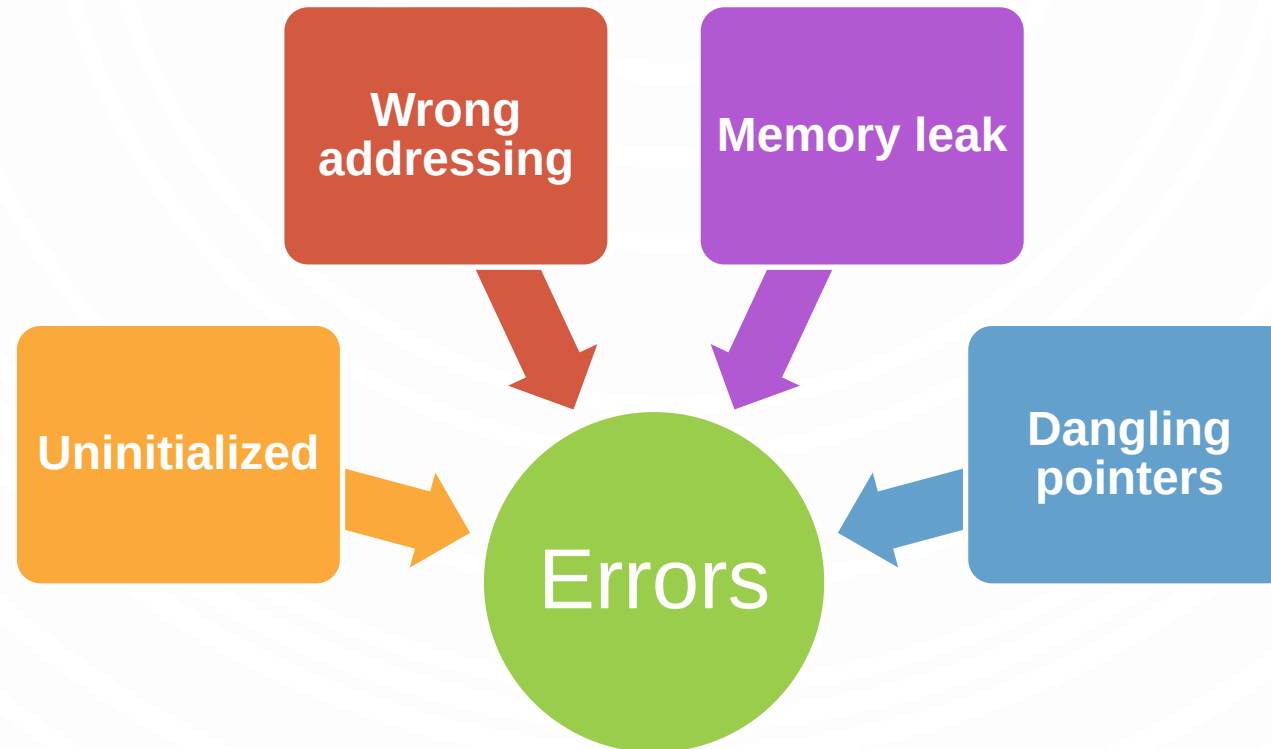


Pointers – Operators

- for a pointer of type T , $T^* pt$, operators $--/++$ add or subtract the address of the pointer with $\text{sizeof}(T)$ bytes;
- for a pointer of type T , $T^* pt$, expression $pt + k / pt - k$, adds or subtracts a number of $k * \text{sizeof}(T)$ bytes, where k is an integer value;
- the difference between two pointers within the same range of values is the number of elements between the two addresses;
- the sum between 2 pointers is not allowed;



Pointers – Frequent errors



Pointers – const modifier

```
int var1 = 7; int var2 = 8;
```

```
// constant pointer to integer
```

```
int * const ptr = &var1;
```

```
ptr = &var2; instead: *ptr = var2;
```

```
// pointer to constant integer
```

```
int const *pint; const int *pint;
```

```
*pint = 4; instead: pint = &var1;
```

```
// constant pointer to constant integer
```

```
const int * const pint = &var;
```



Function pointers syntax:

Declaring: `returned_type (* pointer_name) (args_list);`

Initializing: `pointer_name = function_name;`

Invoking: `pointer_name (args_list);`



Function pointers:

```
int add(int a, int b)
{
    return a+b;
}
int mul(int a, int b)
{
    return a*b;
}
```

Declaration:

```
int (*fct)(int, int);
```

Initialization:

```
fct = add;
```

```
fct = mul;
```

Function pointer call:

```
fct(5,7);
```

